

Good question.

For **MarketState**, there isn't much logic to test because it's just an `Enum`. In real companies, they often **don't write unit tests for a plain enum** unless it contains behavior.

For learning purposes, though, it's fine to have one simple test to verify the values.

Folder Structure

```
backend_engine/
|
├─ market_session/
|   ├── __init__.py
|   └─ market_state.py
|
└─ tests/
    └─ test_market_state.py
```

test_market_state.py

Python

Run

```
from market_session.market_state import MarketState

def test_market_state_values():
    assert MarketState.WAITING.value == "WAITING"
    assert MarketState.OPEN.value == "OPEN"
    assert MarketState.CLOSED.value == "CLOSED"
```

Install pytest

Bash

```
pip install pytest
```

Run all tests

From your project root:

Bash

```
pytest
```

Expected:

```
=====
1 passed
=====
```

Run only one test file

Bash

```
pytest tests/test_market_state.py
```

Why are we writing this?

Not because the enum is likely to fail.

It's to establish your project workflow:

```
Design Module
  ↓
Implement Module
  ↓
Write Test
  ↓
Run Test
  ↓
Git Commit
```

We'll repeat this for every module.

In real companies

For this specific file:

Python

Run

```
class MarketState(Enum):
    WAITING = "WAITING"
```

OPEN = "OPEN"

CLOSED = "CLOSED"

many teams **wouldn't write a unit test** because there is no business logic.

The first module where testing becomes truly valuable is **MarketCalendar**. For example, you'll want to verify:

- Monday → Trading day ✓
- Saturday → Not a trading day ✓
- Sunday → Not a trading day ✓
- Holiday → Not a trading day ✓
- Next trading day after Friday → Monday (or Tuesday if Monday is a holiday) ✓

Those tests validate actual business rules and are where unit testing starts to provide real value.

So my recommendation is:

- ✓ Create the simple enum test if you want to establish the testing pattern.
- 🎯 Focus your testing effort on `MarketCalendar`, `MarketScheduler`, `SubscriptionRegistry`, `CandleBuilder`, `IndicatorEngine`, etc., where the business logic lives. Those modules benefit the most from comprehensive unit tests.